

Python
for DA

Дэшборды в Dash



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  3D графики
-  Graphic Objects
-  Подграфики и множественные оси в Plotly
-  Взаимодействие с графиками в Plotly
-  Анимация в Plotly

План занятия

- Обзор Dash и его экосистемы
- Установка необходимых пакетов для разработки Dash
- Создание простого дашборда
- HTML-компоненты макета
- Интерактивные визуализации данных
- Пример приложения



ОСНОВНОЙ БЛОК





Обзор Dash и его экосистемы



Dash

Это фреймворк, разработанный компанией Plotly для создания интерактивных веб-приложений для работы с данными с помощью Python.

Технологии обеспечения Dash



Функции и возможности



Интерактивность



Простота использования



Настраиваемость



Dash Enterprise

Это расширенная платформа,
предоставляющая дополнительные
возможности для развертывания,
масштабирования и интеграции



ВОПРОСЫ





Установка пакетов для разработки Dash

Установка основного пакета Dash



```
pip install dash
```

Рендерер Dash



```
pip install dash-renderer
```



dash-renderer

Это компонент Dash, который отображает компоненты приложения в веб-браузере.

HTML-компоненты Dash



```
pip install dash-html-components
```

Основные компоненты Dash



```
pip install dash-core-components
```

Запрос установленных пакетов



```
pip show dash
```



ВОПРОСЫ





Создание простого дашборда

Импорт библиотек Dash

```
from dash import Dash, html, dcc  
import plotly.express as px
```

Простейшее приложение Dash

```
# Подключение модулей
from dash import Dash, html, dcc
import plotly.express as px

# Создание экземпляра класса Dash
app = Dash()

# Создание макета приложения
df = px.data.iris()
fig = px.scatter(df, x = 'sepal_width', y = 'sepal_length', color = 'species')
app.layout = html.Div([
    html.H1("Визуализация набора данных Iris"),
    dcc.Graph(figure = fig)
])

# Запуск приложения
app.run()
```



ВОПРОСЫ





HTML-компоненты макета



HTML-контур

Это контур, который обычно включает в себя несколько ключевых разделов: заголовок, навигационная панель, область основного содержимого и нижний колонтитул.



Заголовок

Содержит заголовок панели, а также навигацию верхнего уровня или важную информацию, которую пользователи должны видеть немедленно.

Заголовок

```
header = html.Header(html.H1('My Dashboard'))
```



Навигационная панель

Полезна для размещения навигационных ссылок, фильтров или других интерактивных элементов управления, которые влияют на данные, отображаемые в основной области содержимого.

Боковая панель

```
navbar = html.Div([  
    html.H2('Navigation'),  
    html.Ul([  
        html.Li(html.A('Главная', href='/')),  
        html.Li(html.A('Страница 1', href='/page-1')),  
        html.Li(html.A('Страница 2', href='/page-2'))  
    ])   
])
```



Область основного контента

Это центральная часть приборной панели, где будет отображаться основная информация, визуализации данных или контент.

Область основного контента

```
main_content = html.Section([
    html.Article([
        html.H3('Заголовок основного контента'),
        html.P('Некоторые важные материалы или визуализации данных находятся здесь.')
    ])
])
```




Нижний колонтитул

Содержит информацию об авторских правах, контактные данные или ссылки на соответствующие ресурсы.

Нижний колонтитул

```
footer = html.Footer([  
    html.P('Copyright (c) 2024 My Dashboard'),  
], className='footer')
```

Объединение разделов в макет приложения Dash

```
from dash import Dash, html

app = Dash(__name__)
app.layout = html.Div([
    header,
    sidebar,
    main_content,
    footer
], className='dashboard-container')

if __name__ == '__main__':
    app.run(debug=True)
```



ВОПРОСЫ





Интерактивные визуализации данных



`dash_core_components (dcc)`

Это библиотека в составе фреймворка Dash, которая предоставляет компоненты более высокого уровня, чем HTML-элементы, разработанные специально для интерактивности и визуализации данных.



Graph

Это инструмент в `dash_core_components`, который используется для встраивания диаграмм Plotly в приложения Dash и придания им интерактивности.

Graph



Пример (компонент с пустым графиком)

```
graph_component = dcc.Graph(
    id='example-graph',
    figure={}
)
```

Пояснения

- `id` — Уникальный идентификатор компонента, используемый для ссылок на него.
- `figure` — Словарь, определяющий данные и расположение графика.
 - Ключ `data` содержит список объектов графика
 - ключ `layout` задает общие настройки макета



Dropdown

Это компонент, который позволяет пользователям выбирать из списка опций. Используется для управления отображением данных в других компонентах, например в графиках.

Dropdown



Пример

```
dropdown_component = dcc.Dropdown(
    id='my-dropdown',
    options=[
        {'label': 'San Francisco', 'value':
'SF'},
        {'label': 'Montreal', 'value': 'MTL'}
    ],
    value='SF'
)
```

Пояснения

- `id` — Уникальный идентификатор компонента, используемый для ссылок на него.
- `options` — Список словарей, где каждый словарь представляет опцию с меткой и значением.
- `value` — Выбранное по умолчанию значение.



Slider

Это компонент, который предоставляет пользователям возможность выбрать числовое значение из диапазона.

Slider



Пример

```
slider_component = dcc.Slider(
    min=0,
    max=20,
    step=0.5,
    value=10,
    marks={i: str(i) for i in range(21)}
)
```

Пояснения

- `dcc.Slider` — Создает входной ползунок.
- `min` и `max` — Определяют минимальное и максимальное значения ползунка.
- `step` — Гранулярность каждого шага в слайдере.
- `value` — Значение по умолчанию для ползунка.
- `marks` — Необязательные метки для позиций ползунка, задаваемые в виде словаря.



Обратные вызовы

Прослушивают изменения свойств компонентов и соответствующим образом обновляют другие компоненты.

Обратные вызовы



Пример

```
from dash import Input, Output, callback

@callback(
    Output('example-graph', 'figure'),
    Input('my-dropdown', 'value')
)
def update_graph(selected_value):
    # Логика обновления графика на основе
    # выбранного значения
    pass
```

Пояснения

- @callback — Декоратор для определения функции обратного вызова, которая обновляет части приложения в ответ на пользовательский ввод.
- Output и Input — Укажите компоненты и свойства, которые запускают обратный вызов или обновляются с его помощью.



ВОПРОСЫ





Примеры приложений



1. Использование выпадающего списка

```
from dash import Dash, dcc, html, Input, Output, callback
```

```
# Создаем экземпляр приложения Dash
```

```
app = Dash()
```

```
# Компонент графика
```

```
graph_component = dcc.Graph(
```

```
    id='example-graph',
```

```
    figure={}
)
```

```
# Компонент выпадающего списка
```

```
dropdown_component = dcc.Dropdown(
```

```
    id='my-dropdown',
```

```
    options=[
```

```
        {'label': 'San Francisco', 'value': 'SF'},
```

```
        {'label': 'Montreal', 'value': 'MTL'}
    ],
```

```
    value='SF'
```

```
)
```

начало кода

1. Использование выпадающего списка

Определяем layout приложения

```
app.layout = html.Div([
    dropdown_component,
    graph_component,
])
```

Callback для обновления графика при выборе в выпадающем списке

```
@callback(
    Output('example-graph', 'figure'),
    Input('my-dropdown', 'value')
)
def update_graph(selected_value):
    SF = {'x': [1, 2, 3], 'y': [4, 5, 7], 'type': 'bar', 'name': 'San Francisco'}
    MTL = {'x': [1, 2, 3], 'y': [8, 3, 6], 'type': 'bar', 'name': "Montreal"}
    BERLIN = {'x': [1, 2, 3], 'y': [2, 4, 5], 'type': 'bar', 'name': "Berlin"}
    result = {'data': [eval(selected_value), BERLIN],
              'layout': {'title': 'Dash Data Visualization'}}
    return result
```

```
app.run()
```

продолжение кода

2. Использование слайдера

```
from dash import Dash, dcc, html, Input, Output, callback
```

```
# Создаем экземпляр приложения Dash
```

```
app = Dash(__name__)
```

```
# Компонент графика
```

```
graph_component = dcc.Graph(
```

```
    id='example-graph',
```

```
    figure={},
```

```
)
```

```
# Компонент слайдера
```

```
slider_component = dcc.Slider(
```

```
    id="my_slider",
```

```
    min=0,
```

```
    max=20,
```

```
    step=0.5,
```

```
    value=10,
```

```
    marks={i: str(i) for i in range(21)}
```

```
)
```

начало кода

2. Использование слайдера

Определяем layout приложения

```
app.layout = html.Div([
    graph_component,
    slider_component
])
```

Callback для обновления графика при выборе в выпадающем списке

```
@callback(
    Output('example-graph', 'figure'),
    Input('my_slider', 'value')
)
def update_graph(selected_value):
    return {'data': [
        {'x': [1, 2, 3], 'y': [4, selected_value, 2], 'type': 'line', 'name': "SF"},
        {'x': [1, 2, 3], 'y': [2, 4, selected_value//2+4], 'type': 'line', 'name': "MTL"}
    ],
        'layout': {
            'title': 'Dash Data Visualization'
        }
    }
```

```
app.run()
```

продолжение кода

3. Использование радиокнопок

```

from dash import Dash, html, dcc, callback, Output, Input
import plotly.express as px

df = px.data.iris()

app = Dash()

app.layout = [
    html.Div(html.H2('My First App with Graph and Controls')),
    dcc.RadioItems(options=['sepal_width', 'petal_length', 'petal_width'],
value='sepal_width', id='controls-and-radio-item'),
    dcc.Graph(figure={}, id='controls-and-graph')
]

@callback(
    Output(component_id='controls-and-graph', component_property='figure'),
    Input(component_id='controls-and-radio-item', component_property='value')
)
def update_graph(col_chosen):
    fig = px.scatter(df, x=col_chosen, y='sepal_length', color='species')
    return fig

app.run()

```



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание

Что делает следующий фрагмент кода?

```
from dash import Dash, html, dcc
import plotly.express as px

app = Dash(__name__)

app.layout = html.Div([
    html.H1("Визуализация набора данных Iris"),
    dcc.Graph(figure=px.scatter(px.data.iris(), x='sepal_width', y='sepal_length',
color='species')),
    html.P("Настройте ползунок, чтобы изменить график:"),
    dcc.Slider(min=0, max=10, step=0.5, value=5)
])

app.run(mode='inline')
```




ВОПРОСЫ



Домашнее задание

Создайте простое приложение Dash, которое позволяет пользователям выбрать континент из выпадающего меню и динамически отображает гистограмму ожидаемой продолжительности жизни для выбранного континента, используя набор данных Gapminder.

1. Инициализируйте приложение Dash.
2. Определите макет, включающий `dcc.Dropdown` для выбора континента и `dcc.Graph` для отображения гистограммы.
3. Наполните выпадающие опции континентами из набора данных Gapminder.
4. Напишите функцию обратного вызова, которая обновляет гистограмму в зависимости от выбранного континента.
5. Гистограмма должна отображать распределение `lifeExp` для выбранного континента.
6. Добавьте к гистограмме соответствующие заголовки и метки осей.

Заключение

